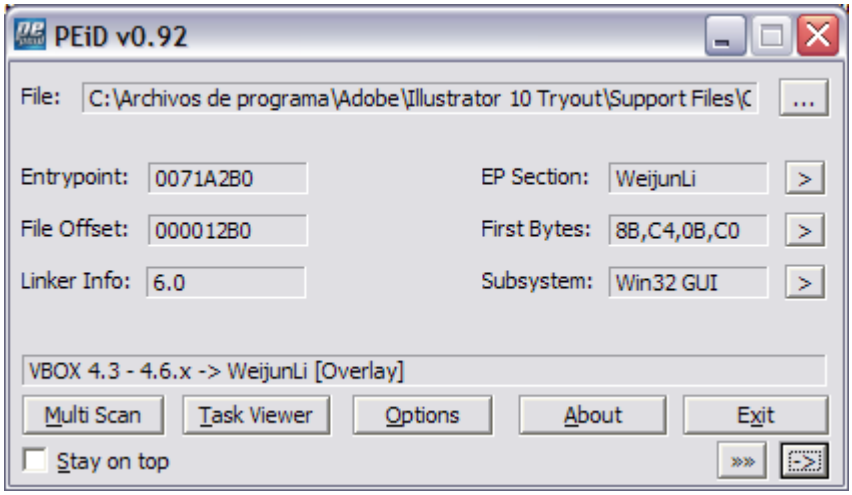
Programa:	Adobe Illustrator 10	
Protección:	Empaquetado con Vbox y Trial 30 días	
Objetivos:	Desempaquetar el archivo (el Trial cae directo al desempakar, jeje..)	
URL descarga:	---	
Dificultad:	Sabiendo lo que es la IAT no problema… ;DDD	
Herramientas:	OllyDbg + PEiD + LordPE + ImpRec.	
Cracker:	SyXe'05 [hAcK-c0d3d]	Fecha: 25-07-2005

Hola chicos!! Aquí estamos de vuelta con otra protección comercial de las que están de moda parece, jeje.. algunas compañías se han hecho adictas del empaquetado y lanzan sus aplicaciones bien armadas de puñetas pero igual nosotros nos las arreglaremos para vencer estos truquitos, hoy trataremos con uno de esos programas concretamente con la versión 10 del ***Adobe Illustrator***, supongo que todos lo conoceremos pues ya van por la versión ***1x*** ¿? bien como siempre lo abriremos con ***PEiD*** a ver qué dice, pues dice ***Nothing found [Overlay] ****** para mirar 'más profundo' picamos en la pestañita de la flecha y seleccionamos ***Hardcore Scan*** y entonces ahora sí aparecerá esto:



Ahí está: ***VBOX 4.3 - 4.6.x -> WeijunLi [Overlay]*** ahora ya sabemos a que nos enfrentamos, esto es fundamental ya que nuestros ataques se planificarán de distinta manera ante un Armadillo que ante un ***ASProtect***, ***Vbox***, etc., es bueno conocer bien a nuestra víctima y una buena información obtenida en la primera fase puede ser determinante para el crack final tened esto muy en cuenta, el método de trabajo de ***Vbox*** es el siguiente: Nosotros lanzamos la aplicación, en este caso ***Illustrator.exe*** este en un momento determinado le pasa el control a ***Vbox#.dll*** (***#*** es variable) que es quien realmente se encarga de la protección temporal, de mostrar la nag, etc., en definitiva la que realmente protege al programa principal, pues al tomar el control esta dll verifica si nos queda tiempo de evaluación disponible y que este no haya expirado, de ser asi muestra la nag con el botón ***Try*** habilitado, al pulsarlo "arma" la ***IAT***, desencripta/desempaka la sección ***.code*** y luego salta al ***OEP*** en una instrucción del tipo:

```
MOV EBX, DWORD PTR [EBP-XX]    ;Aquí está el OEP en [EBP-XX]
JMP EBX                        ;Este es el salto buscado
```

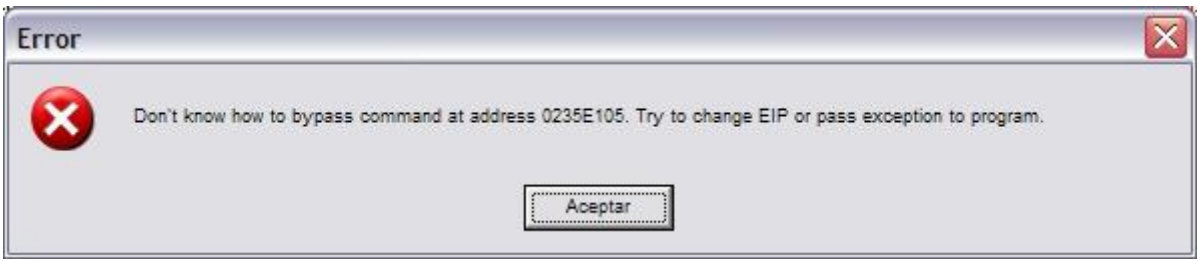
Probad una cosa ya veréis lo que os digo: anotad el ***OEP*** del programa, cuando expire saldrá la nag con el botón ***Try*** deshabilitado como todos sabemos, nada llegar al salto que va al ***OEP*** y que habremos anotado su dirección, notad que esta seguramente variará de una ejecución a otra pues ***Vbox*** carga el código aleatoriamente en memoria aunque no variará mucho, lo importante es que lleguéis al salto que va al ***OEP***, veréis esto:

```
MOV EBX, DWORD PTR [EBP-XX]    ;Aquí está Exit_Process() en [EBP-XX]
JMP EBX                        ;Salta a Exit_Process()
```

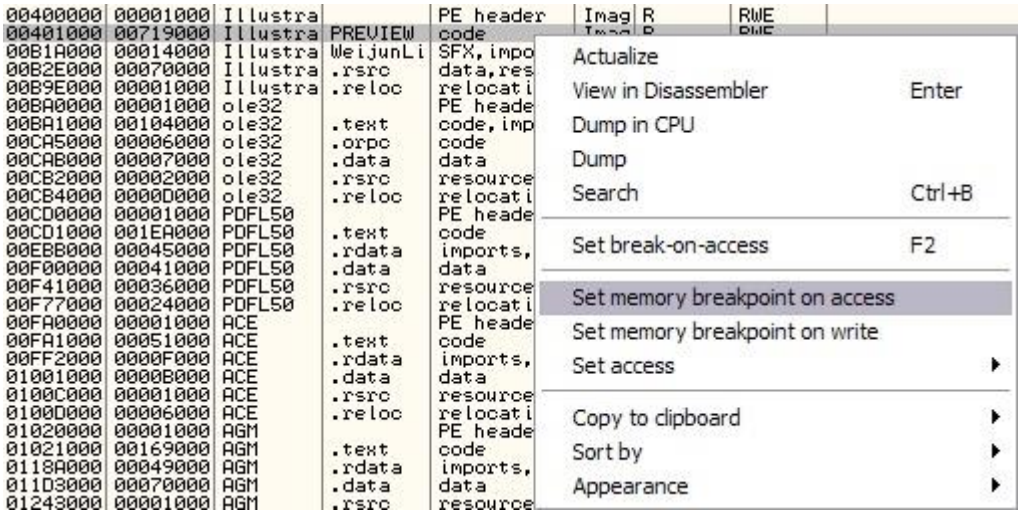
Vemos que nos va a tirar a la calle, ahora nos situamos sobre el ***JMP EBX*** y le damos a ***EBX*** precisamente el valor del ***OEP*** que ya sabemos de antes, saltaremos allí pero **¿?** Qué pasa donde está el ***PUSH EBP*** que debía estar allí? No hay nada más que basura! Esto es lo que os quería enseñar, Vbox no ha desencriptado/desempaquetado esta sección ya que no nos quedaba tiempo Trial así que nuestro intento de llegar

al ***OEP*** no a tenido éxito, para que este hubiera estado correcto deberíamos haber atacado la comprobación temporal dentro de la propia dll justo a la salida de la nagscreen, pero ojo que no es nada sencillo almenos para mi gusto evitar el Time-Trial, pues muchos saltos nos llevan a '***Error Vbox inicialization***' o algo así, lo mejor entonces es o bien comerse el tarro y evitar esto o bien atacar el programa cuando todavía no ha expirado, pues esto nos ahorrará muchos quebraderos de cabeza, jeje..

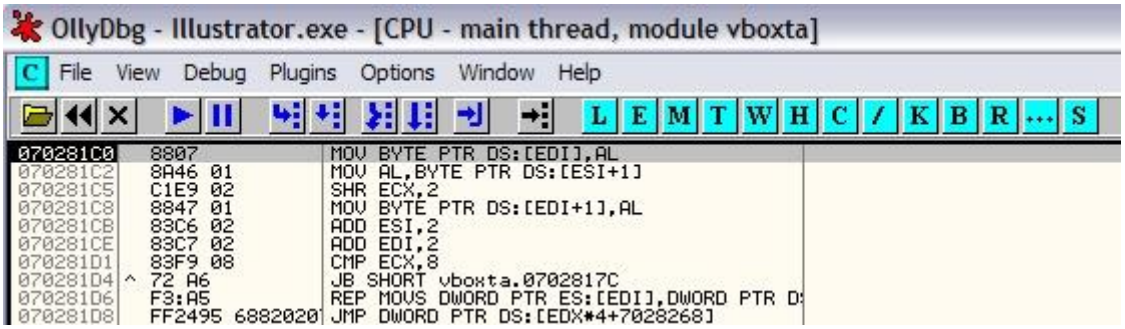
Y dicho esto sigamos a lo nuestro, ya que sabemos a grandes rasgos como trabaja este packer lo que haremos aquí será lo siguiente: estando en el ***EP*** => ***:00B1A2B0*** le damos a ***RUN*** (***F9***) para dejar correr la aplicación, aparecerán dos mensajes que indican que las dll's que esta cargando tienen el ***EP*** fuera de la sección ***.code*** lo que indica que casi con total seguridad están empaquetadas (con ***Vbox*** por cierto, jeje) nada aceptamos y sale esto:



No le hacemos ni caso aceptamos y le damos a ***Shift+F9*** y seguimos… saldrá la nag en pantalla, ahora vamos a la sección de memoria ***Alt+M*** y ponemos un BPM en la primera sección como se ve en la siguiente imagen:



Ahora vamos al escritorio y pulsamos ***Try*** en la nag del programa y ***Olly*** parara en esta zona:



Si miramos en la barra de titulo vemos que estamos en ***vboxta.dll***, veamos porque paro ***Olly***:

Ahí esta la información que nos da en la barra de estado indica que esta ***escribiendo*** en esta sección, esto no nos interesa para nada, si vamos pulsando ***F9*** al poco para aquí:

```
OllyDbg - Illustrator.exe - [CPU - main thread, module vboxta]
0700B17A   50              PUSH EAX
0700B17B   E8 26FBFFFF     CALL vboxta.0700ACA6
0700B180   83C4 14         ADD ESP,14
0700B183   85C0            TEST EAX,EAX
0700B185   0F84 80000000   JE vboxta.0700B20B
0700B18B   FF45 F0         INC DWORD PTR SS:[EBP-10]
0700B18E   8906            MOV DWORD PTR DS:[ESI],EAX          COOLTYPE.CTFindFont
0700B190   8B45 F0         MOV EAX,DWORD PTR SS:[EBP-10]
0700B193   83C7 04         ADD EDI,4
0700B196   3B45 D8         CMP EAX,DWORD PTR SS:[EBP-28]
```

Esta rellenando la ***Iat***, jeje.. mira por donde le hemos pillado el sitio, miremos los registros a ver que dicen:

```
Registers (3DNow!)
EAX 10057BB5 COOLTYPE.CTFindFont
ECX 0013EA80
EDX 77FB306C ntdll.77FB306C
EBX 00000001
ESP 0013E9B4 ASCII "CoolType.dll"
EBP 0013EA8C
ESI 00974064 Illustra.00974064
EDI 00000064

EIP 0700B18E vboxta.0700B18E
```

Son más que delatores, está montando la Iat sin duda alguna, no ha hecho falta ni seguir ningún puntero ni nada hemos llegado como caídos del cielo ;DDD esto me esta empezando a gustar, ya que estamos aquí echamos un vistazo a la zona en busca del mágico pero no lo encontramos ¿? Entonces sigamos hasta el ***OEP***, como el ***BPM*** no para de saltar lo deshabilitamos y vamos al siguiente ret con ***Ctrl+F9***, superamos el ret y volvemos a poner el ***BPM*** en la primera sección pero igualmente salta muchas veces, lo quitamos nuevamente y realizamos la misma jugada, superamos el siguiente ret y lo ponemos otra vez, cuidado que el ***OEP*** no esta muy lejos, en una de estas veces ***Olly*** romperá justo aquí:

```
OllyDbg - Illustrator.exe - [CPU - main thread, module Illustra]
009292EE   55      DB 55      CHAR 'U'
009292EF   8B      DB 8B
009292F0   EC      DB EC
009292F1   6A      DB 6A      CHAR 'j'
009292F2   FF      DB FF
009292F3   68      DB 68      CHAR 'h'
009292F4   78      DB 78      CHAR 'x'
009292F5   DD      DB DD
009292F6   97      DB 97
009292F7   00      DB 00
```

No se aprecia muy bien, veamos porque paro, en la barra de estado (debajo del todo) vemos esto:

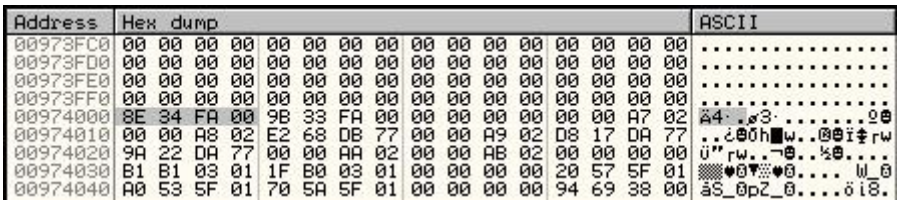
```
Hardware breakpoint 2 at Illustra.009292EE
```

Pues paró por ***ejecución*** esto hay que mirarlo bien entonces, picamos con el botón derecho en la ventana de código y seleccionamos ***Analisys / Remove analisys from*** module, por arte de magia aparece nuestro ***OEP***:

```
OllyDbg - Illustrator.exe - [CPU - main thread, module Illustra]
009292EE   55              PUSH EBP
009292EF   8BEC            MOV EBP,ESP
009292F1   6A FF           PUSH -1
009292F3   68 78DD9700     PUSH Illustra.0097DD78
009292F8   68 F0729200     PUSH Illustra.009272F0
009292FD   64:A1 00000000  MOV EAX,DWORD PTR FS:[0]
00929303   50              PUSH EAX
00929304   64:8925 000000  MOV DWORD PTR FS:[0],ESP
0092930B   83EC 58         SUB ESP,58
0092930E   53              PUSH EBX
0092930F   56              PUSH ESI
00929310   57              PUSH EDI
00929311   8965 E8         MOV DWORD PTR SS:[EBP-18],ESP
00929314   FF15 24439700   CALL DWORD PTR DS:[974324]         kernel32.GetVersion
0092931A   33D2            XOR EDX,EDX
0092931C   8AD4            MOV DL,AH
```

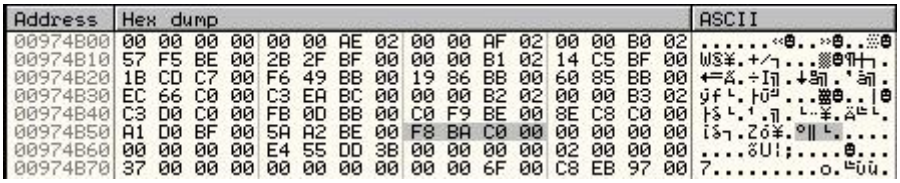
Ya tenemos el primer dato => ***OEP*: 9292EE**

Aprovechamos que estamos aquí para dumpear con LordPE e intellidump activado y para buscar la ***IAT*** que en este caso no es difícil pues si nos situamos sobre el call que va a ***GetVersion*** que esta en la imagen y hacemos ***Follow in Dump*** caeremos en plena ***IAT***, a partir de aquí hay que tener un poco de vista para ver donde empieza y donde acaba la tabla, veamos:



Address	Hex dump	ASCII
00973FC0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00973FD0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00973FE0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00973FF0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00974000	8E 34 FA 00 9B 33 FA 00 00 00 00 00 00 00 A7 02	Ä4·.ø3·.....º☻
00974010	00 00 A8 02 E2 68 DB 77 00 00 A9 02 D8 17 DA 77	..¿☻ôh▄w..⌐☻ï‡rw
00974020	9A 22 DA 77 00 00 AA 02 00 00 AB 02 00 00 00 00	ü"rw..¬☻..½☻....
00974030	B1 B1 03 01 1F B0 03 01 00 00 00 00 20 57 5F 01	▒▒♥☺▼▒♥☺.... W_☺
00974040	A0 53 5F 01 70 5A 5F 01 00 00 00 00 94 69 38 00	áS_☺pZ_☺....öi8.

Aquí esta el inicio de la tabla, vamos bajando hasta buscar el final de la misma:



Address	Hex dump	ASCII
00974B00	00 00 00 00 00 00 AE 02 00 00 AF 02 00 00 B0 02	.....«☻..»☻..▒☻
00974B10	57 F5 BE 00 2B 2F BF 00 00 00 B1 02 14 C5 BF 00	Wß¥.+/┐...▒☻¶├┐.
00974B20	1B CD C7 00 F6 49 BB 00 19 86 BB 00 60 85 BB 00	←═Ä.÷I╗.↓å╗.`à╗.
00974B30	EC 66 C0 00 C3 EA BC 00 00 00 B2 02 00 00 B3 02	∞f└.├ê╝...▓☻..│☻
00974B40	C3 D0 C0 00 FB 0D BB 00 C0 F9 BE 00 8E C8 C0 00	├╨└.╠♪╗.└ù¥.Ä╚└.
00974B50	A1 D0 BF 00 5A A2 BE 00 F8 BA C0 00 00 00 00 00	í╨┐.Zó¥.°╢└.....
00974B60	00 00 00 00 E4 55 DD 3B 00 00 00 00 02 00 00 00	....ßUÝ;....☻...
00974B70	37 00 00 00 00 00 00 00 00 00 6F 00 C8 EB 97 00	7.........o.╚ëù.

Pues allí está el final también, los bytes seleccionados en la imagen representan el último puntero así que le daremos el siguiente como el final de la Iat, si hay algún puntero que presentara ciertas dudas una manera de despejarlas sería seleccionar los bytes de la entrada y picar ***Ctrl+R*** aparecerá esto:

References in Illustra:PREVIEW to 00974030..00974033

Address	Disassembly	Comment
008FD99A	JMP DWORD PTR DS:[974030]	AGM.AGMTerminate

Esta sería válida ya que apunta a una ***API*** correcta, así podríamos seguir con todas las que nos planteen dudas, hecho esto nos quedarán entonces los siguientes datos:

IAT => Inicio: 974000
Final: 974b5c
Largo: b5c

Ahora que tenemos todos estos datos viene la parte que puede presentarnos más problemas, es un paso sencillo pero a la vez delicado, vamos a injertar código que nos ayude con la reparación de la tabla e injertaremos a ***vboxta.dll*** al más puro estilo ***Ricardo Narvaja***, jeje.. saludos compañero eres un fenómeno! Como siempre adoptaremos uno de sus métodos para reparar la tabla, estando parados en el ***OEP*** busquemos una zona libre donde sólo haya ceros, ésta aparece a partir de ***:00B19500***, vamos allí e injertamos esto:

Injerto:

```
:00B19500   C705 FC94B100 0>  MOV DWORD PTR DS:[B194FC],Illustra.00974000
:00B1950A    A1 FC94B100      MOV EAX,DWORD PTR DS:[B194FC]
:00B1950F   8B00              MOV EAX,DWORD PTR DS:[EAX]
:00B19511   3D 00000001       CMP EAX,1000000
:00B19516   7C 14             JL SHORT Illustra.00B1952C
:00B19518   3D 00000003       CMP EAX,3000000
:00B1951D    77 0D            JA SHORT Illustra.00B1952C
:00B1951F   FFD0              CALL EAX
:00B19521   8B1D FC94B100     MOV EBX,DWORD PTR DS:[B194FC]
:00B19527   8903              MOV DWORD PTR DS:[EBX],EAX
:00B19529   83C3 04           ADD EBX,4
```

```
:00B1952C    8305 FC94B100 0>  ADD DWORD PTR DS:[B194FC],4
:00B19533    813D FC94B100 4>  CMP DWORD PTR DS:[B194FC],Illustra.00974B5C
:00B1953D    74 02             JE SHORT Illustra.00B19541
:00B1953F  ^ EB C9             JMP SHORT Illustra.00B1950A
:00B19541    90                NOP      <= BPX Aqui...!!!
```

<u>En binario</u>:

C7 05 FC 94 B1 00 00 40 97 00 A1 FC 94 B1 00 8B 00 3D 00 00 00 01 7C 14 3D 00 00 00 03 77 0D FF D0 8B 1D FC 94 B1 00 89 03 83 C3 04 83 05 FC 94 B1 00 04 81 3D FC 94 B1 00 5C 4B 97 00 74 02 EB C9 90 90

Tenemos que injertar un poquitín de código en la librería donde se obtiene la ***API*** correcta en este caso se trata de ***vboxta.dll***, vamos allí e injertamos este código a partir de ***:0700E5F7*** que es donde ***EAX*** ya tiene el valor ***API*** correcto que obtuvo del call anterior, ahora saltamos a nuestra rutina para guardarlo en la Iat, jeje..

<u>En Vboxta.dll</u>:

```
:0700E5F7    68 2195B100       PUSH 0B19521
:0700E5FC    C3                RETN
```

<u>En binario</u>:

68 21 95 B1 00 C3

Situamos el ***EIP*** al inicio del injerto ***:00B19500*** y le damos a ***F9*** Olly parará por error, si nos fijamos estamos en una excepción que parece insuperable, esto es debido a que el injerto es algo débil y pasan por el filtro punteros que no deberían pasar, entonces la rutina intentará procesarlos como punteros inválidos generándose un error tipo '***Acces Violation'*** llegando a ***'RaiseException'***, por tanto la consiguiente detención del programa, para que esto no ocurra echamos un vistazo a la tabla y pondremos unos límites para que no se procesen estos punteros, veamos

Punteros válidos – intervalos (y que engañan a nuestra rutina):

974030 – 974184
9744F4 – 974870

Estos son los dos intervalos que la rutina debe saltar, veamos una imagen de ejemplo:

Address	Hex dump															
00974030	B1	B1	03	01	1F	B0	03	01	00	00	00	00	20	57	5F	01
00974040	A0	53	5F	01	70	5A	5F	01	00	00	00	00	94	69	38	00
00974050	A8	66	38	00	00	00	00	00	30	56	3B	00	F0	54	3B	00
00974060	00	00	00	00	B5	7B	05	10	5B	2A	05	10	95	28	05	10
00974070	27	C0	07	10	70	BF	07	10	08	BF	07	10	E5	BE	07	10
00974080	AB	BE	07	10	18	BE	07	10	BD	20	05	10	84	20	05	10
00974090	54	2E	05	10	13	2D	05	10	41	D3	07	10	25	D3	07	10
009740A0	0D	D3	07	10	C0	D2	07	10	82	D2	07	10	4A	D2	07	10

Estos punteros ya son validos y no hay que tocarlos, esto es fácil de conseguir añadiendo unas líneas de código a nuestra rutinilla que detecten cuando llegamos a estas direcciones y las salte, por ejemplo así:

Al llegar a :974030 => MOV [FLAG], 974188 **; Saltamos la primera zona**
Al llegar a :9744F4 => MOV [FLAG], 974874 **; Saltamos la segunda zona**

En una especie de pseudocódigo sería algo así:

```
MOV DWORD PTR DS:[B194FC],Illustra.00974000
MOV EAX,DWORD PTR DS:[B194FC]

CMP EAX, 974030
JE FLAG1
CMP EAX, 9744F4
JE FLAG2

MOV EAX,DWORD PTR DS:[EAX]
```

```
CMP EAX,1000000
JL SHORT Illustra.00B1952C
CMP EAX,3000000
JA SHORT Illustra.00B1952C
CALL EAX
MOV EBX,DWORD PTR DS:[B194FC]
MOV DWORD PTR DS:[EBX],EAX
ADD EBX,4
ADD DWORD PTR DS:[B194FC],4
CMP DWORD PTR DS:[B194FC],Illustra.00974B5C
JE SHORT Illustra.00B19541
JMP SHORT Illustra.00B1950A
NOP

FLAG1:
MOV DWORD PTR [0B194FC], 974188
JMP 00B1950A

FLAG2:
MOV DWORD PTR [0B194FC], 974874
JMP 00B1950A
```

Esto pasado a lenguaje Ensamblador daría algo así:

Rutina_Final :

```
:00B19500   C705 FC94B100 0>  MOV DWORD PTR DS:[B194FC],Illustra.00974000   ; inicio de la IAT
:00B1950A   A1 FC94B100       MOV EAX,DWORD PTR DS:[B194FC]                 ; en EAX la posición IAT
:00B1950F   3D 30409700       CMP EAX,Illustra.00974030                     ; modificamos el EIP?
:00B19514   74 3A             JE SHORT Illustra.00B19550                    ; SI => Vamos...
:00B19516   3D F4449700       CMP EAX,Illustra.009744F4                     ; modificamos el EIP?
:00B1951B   74 3F             JE SHORT Illustra.00B1955C                    ; SI => Vamos...
:00B1951D   8B00              MOV EAX,DWORD PTR DS:[EAX]                    ; en EAX la posible API
:00B1951F   3D 00000001       CMP EAX,1000000                               ; esta redirigida?
:00B19524   7C 11             JL SHORT Illustra.00B19537                    ; NO => no la procesamos
:00B19526   3D 00000003       CMP EAX,3000000                               ; esta redirigida?
:00B1952B   77 0A             JA SHORT Illustra.00B19537                    ; NO => no la procesamos
:00B1952D   FFD0              CALL EAX                                      ; obtenemos la API
:00B1952F   8B1D FC94B100     MOV EBX,DWORD PTR DS:[B194FC]                 ; en EBX la posición IAT
:00B19535   8903              MOV DWORD PTR DS:[EBX],EAX                    ; ponemos la API en la IAT
:00B19537   83C3 04           ADD EBX,4                                     ; siguiente puntero (sobra)
:00B1953A   8305 FC94B100 0>  ADD DWORD PTR DS:[B194FC],4                   ; siguiente puntero
:00B19541   813D FC94B100 4>  CMP DWORD PTR DS:[B194FC],Illustra.00974B5C   ; hemos terminado?
:00B1954B   74 02             JE SHORT Illustra.00B1954F                    ; SI => Salimos...
:00B1954D   EB BB             JMP SHORT Illustra.00B1950A                   ; NO => Seguimos...
:00B1954F   90                NOP      <= BPX Aqui...!!!                    ; Aquí parara al acabar
:00B19550   C705 FC94B100 8>  MOV DWORD PTR DS:[B194FC],Illustra.00974184   ; Pasamos el filtro_1
:00B1955A   EB AE             JMP SHORT Illustra.00B1950A                   ; Seguimos...
:00B1955C   C705 FC94B100 7>  MOV DWORD PTR DS:[B194FC],Illustra.00974874   ; Pasamos el filtro_2
:00B19566   EB A2             JMP SHORT Illustra.00B1950A                   ; Seguimos...
:00B19568   90                NOP
```

En binario :

C7 05 FC 94 B1 00 00 40 97 00 A1 FC 94 B1 00 3D 30 40 97 00 74 3A 3D F4 44 97 00 74 3F 8B 00 3D 00 00 00 01 7C 11 3D 00 00 00 03 77 0A FF D0 8B 1D FC 94 B1 00 89 03 83 C3 04 83 05 FC 94 B1 00 04 81 3D FC 94 B1 00 5C 4B 97 00 74 02 EB BB 90 C7 05 FC 94 B1 00 84 41 97 00 EB AE C7 05 FC 94 B1 00 74 48 97 00 EB A2 90

Veamos como deberá quedar una vez escrito en Olly:

```
OllyDbg - Illustrator.exe - [CPU - main thread, module Illustra]
File View Debug Plugins Options Window Help

00B1952D   FFD0          CALL EAX
00B1952F   8B1D FC94B100 MOV EBX,DWORD PTR DS:[B194FC]
00B19535   8903          MOV DWORD PTR DS:[EBX],EAX
00B19537   83C3 04       ADD EBX,4
00B1953A   8305 FC94B100 ADD DWORD PTR DS:[B194FC],4
00B19541   813D FC94B100 CMP DWORD PTR DS:[B194FC],Illustra.00974B5C
00B1954B v 74 02         JE SHORT Illustra.00B1954F
00B1954D ^ EB BB         JMP SHORT Illustra.00B1950A
00B1954F   90            NOP
00B19550   C705 FC94B100 MOV DWORD PTR DS:[B194FC],Illustra.00974184
00B1955A ^ EB AE         JMP SHORT Illustra.00B1950A
00B1955C   C705 FC94B100 MOV DWORD PTR DS:[B194FC],Illustra.00974874
00B19566 ^ EB A2         JMP SHORT Illustra.00B1950A
00B19568   90            NOP
00B19569   0000          ADD BYTE PTR DS:[EAX],AL
```

Esto en cuanto a ***Illustrator.exe***, ahora para ***vboxta.dll*** quedará de esta manera:

```
0700E5EE   50            PUSH EAX
0700E5EF   E8 12000000   CALL vboxta.0700E606
0700E5F4   68 2F95B100   PUSH 0B1952F
0700E5F9   C3            RETN
0700E5FA   8B5D F8       MOV EBX,DWORD PTR SS:[EBP-8]
0700E5FD   8B4D F4       MOV ECX,DWORD PTR SS:[EBP-C]
0700E600   8B55 F0       MOV EDX,DWORD PTR SS:[EBP-10]
```

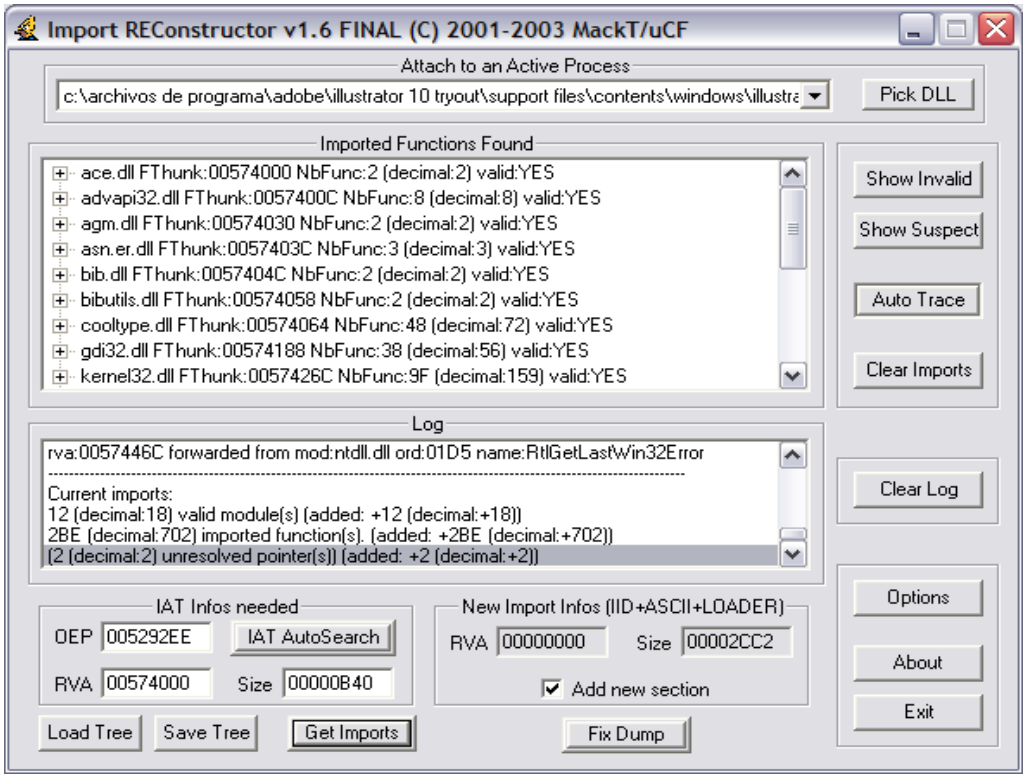
Así está perfecto, después no olvidar el ***BPX*** en el ***NOP*** para que ***Olly*** pare al terminar de reparar la tabla entera, a estas alturas estamos muy cerca del desempaquetado final, jeje.. si habéis seguido todos estos pasos no tendréis problemas estoy convencido, veamos como quedó la tabla, vamos a ver una imagen del final de la misma:

```
Address   Hex dump                                          ASCII
00974AC0  00 00 00 00 E3 15 BD 77 2D 16 BD 77 7D 16 BD 77  ....ò$¢w-_¢wJ_¢w
00974AD0  00 00 00 00 AD 5F F8 72 69 5E F8 72 63 62 F8 72  ....¡_°ri^°rcb°r
00974AE0  6A 5D F8 72 63 16 F8 72 00 00 00 00 7B 42 37 76  j]°rc_°r....{B7v
00974AF0  BF 6F 36 76 05 31 37 76 90 23 36 76 CF 90 36 76  ┐o6v♣17vÉ#6v╧É6v
00974B00  00 00 00 00 9F E1 BC 00 44 30 C1 00 77 21 BB 00  ....ƒßª.D0┴.w!╗.
00974B10  57 F5 BE 00 2B 2F BF 00 C2 0C BC 00 14 C5 BF 00  W§¥.+/┐.┬.♀.¶┼┐.
00974B20  1B CD C7 00 F6 49 BB 00 19 86 BB 00 60 85 BB 00  ←═Ä.÷I╗.↓å╗.`à╗.
00974B30  EC 66 C0 00 C3 EA BC 00 F4 83 BB 00 58 ED BC 00  ∞f└.├Ûª.¶â╗.XYª.
00974B40  C3 D0 C0 00 FB 0D BB 00 C0 F9 BE 00 8E C8 C0 00  ├╨└.√♪╗.└·¥.ÄÈ└.
00974B50  A1 D0 BF 00 5A A2 BE 00 F8 BA C0 00 00 00 00 00  í╨┐.Z¢¥.°╢└.....
```

Allí se ven los valores en rojo son los que reparó el injerto y que eran punteros inválidos inicialmente

Ahí esta explicado el tema el injerto con comentarios al margen que indican a grandes rasgos su comportamiento que es muy sencillo, el que quiera mas info que me escriba o bien que pille los tutos de Vbox de Ricardo que están en el ***FTP*** ya que el es el autor del injerto aquí solo le he añadido dos cositas para evitar que se procesen ciertas partes de la tabla ya que son punteros que apuntan a zonas del tipo ***:01xxxxxx*** que son muy conflictivas ya que están en el mismo intervalo que las ***API's*** redirigidas que están entre ***:01000000*** y ***:03000000*** entonces nuestra rutina (bueno la de ***Ricardo***, jeje) intentará procesar estos punteros pero como ya son validos esto generará un error, esto lo evitamos poniendo dos filtros que eviten esto, simplemente llevara un control del ***EIP*** para saber donde estamos en caso de estar en el inicio de los bloques conflictivos pues los saltara y continuaremos como si nada, como veis es muy sencillito su comportamiento pero muy eficaz ya que los resultados son espectaculares como vimos en la imagen anterior ;DDD

Es el momento de darle su parte de protagonismo al ***ImpRec***, pongamos los valores correctos de ***OEP***; ***IAT*** y el largo de la misma, luego a "***Get Imports***" y saldrá:



Quedaron sólo dos entradas sin resolver, veámoslas:

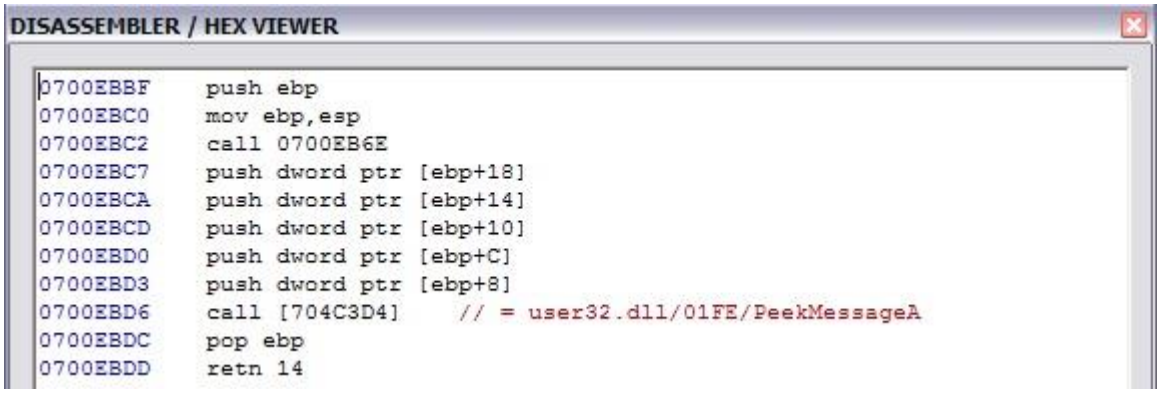
<u>Entrada1</u>:

IAT rva: 5748A0
Apunta a: 700EB50

API Correcta: GetMessageA

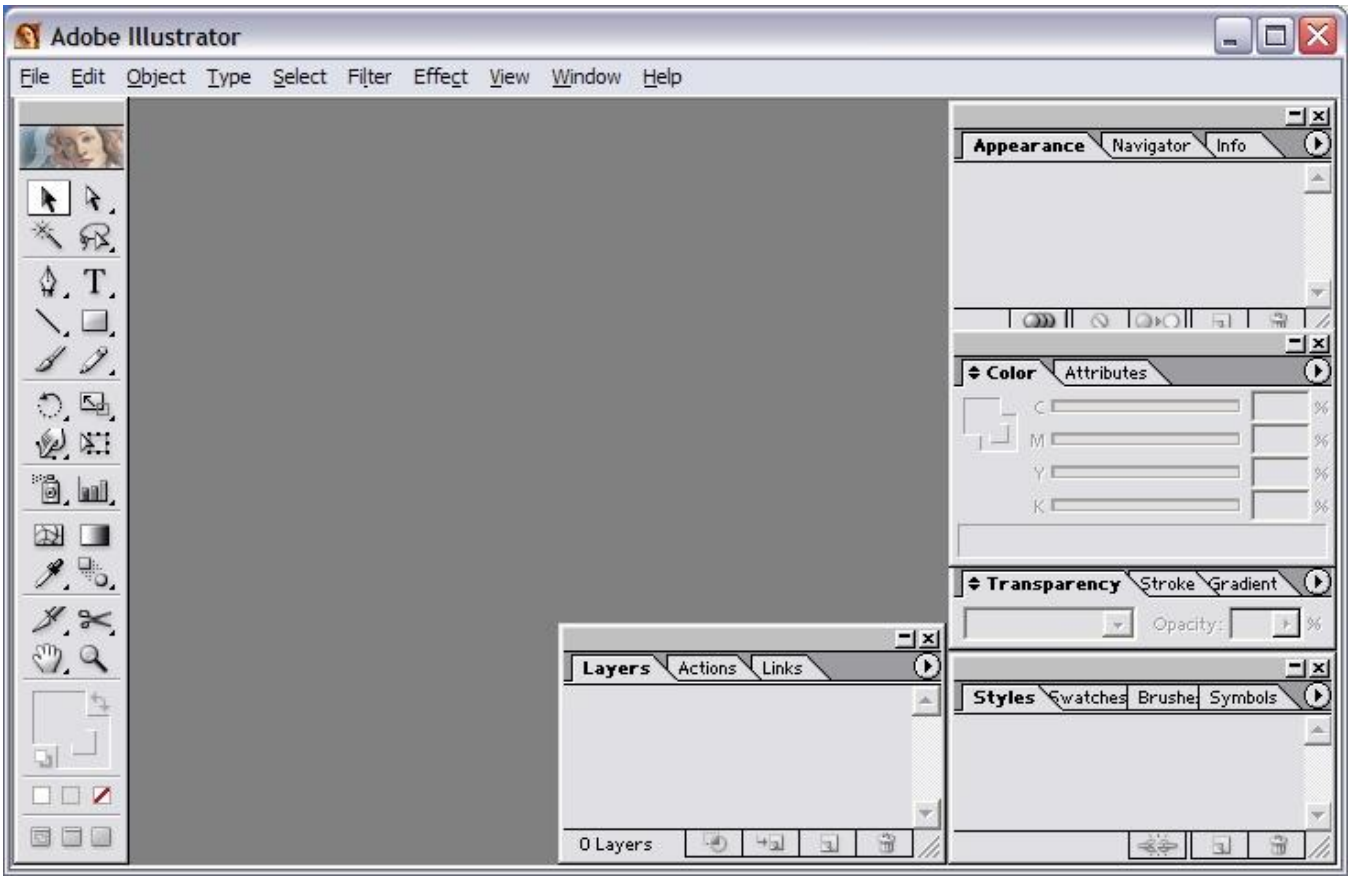
<u>Entrada2</u>:

IAT rva: 5749FC
Apunta a: 700EBBF



API Correcta: PeekMessageA

Reparamos estas dos manualmente desde el propio Import luego a "***Fix Dump***" lo probamos y…



Funciona…..!!!

Ya tenemos la aplicación desvboxeada, jeje.. decir que podemos aplicar este magnífico método de Ricardo para reparar la Iat en cualquier programa protegido con Vbox ;DDDD como ya sabemos no debemos preocuparnos del Time-Trial ya que lo llevaba el propio packer y no la aplicación en si, o sea que hemos matado dos pájaros de un tiro, jeje..

Y por fin con esto damos por finalizado el tuto, espero que os haya gustado, pronto volveremos con otros, si alguien quiere contactar conmigo por cualquier tema, ya sea una duda, una sugerencia o como no una crítica siéntase libre de escribirme a cualquiera de estas direcciones:

syxe00@yahoo.es
syxe00@hotmail.com